

ĐỀ XUẤT CÔNG NGHỆ & CẤU TRÚC THƯ MỤC DỰ ÁN

Nền tảng website tuyển dụng ứng dụng AI
Tech stack • Kiến trúc thư mục Backend/Frontend • Lớp tích hợp AI/LLM

Tài liệu bổ sung — Phiên bản 1.0

1. LỰA CHỌN CÔNG NGHỆ THEO TỪNG LỚP

Lớp	Công nghệ đề xuất	Lý do lựa chọn
Backend	Python — FastAPI	Async tốt, dễ tích hợp AI/LLM, type-safe với Pydantic, hệ sinh thái AI phong phú (LangChain, Anthropic/OpenAI SDK)
Frontend	Next.js (React) + TypeScript	SSR cho trang JD công khai (tốt cho SEO); tách rõ App HR (dashboard) và Public site (ứng viên)
Database chính	PostgreSQL	Quan hệ rõ ràng; hỗ trợ mở rộng pgvector để làm semantic search mà chưa cần Vector DB riêng ở giai đoạn đầu
Cache / Queue	Redis + Celery (hoặc RQ)	Xử lý nền: chấm điểm CV, Speech-to-Text ghi âm, gửi mail hàng loạt — không chặn luồng request chính
File Storage	S3-compatible (AWS S3 / MinIO)	Lưu trữ CV, file ghi âm, transcript với khả năng mở rộng và phân quyền truy cập
Xác thực (Auth)	JWT + OAuth2 (hoặc Auth0/Clerk)	Hỗ trợ multi-tenant (nhiều công ty) và phân quyền theo vai trò trong từng công ty
Dịch vụ Email	SendGrid / Amazon SES	Gửi email hàng loạt có theo dõi trạng thái mở/click, tỷ lệ deliverability cao
LLM Provider	Anthropic Claude API (qua lớp abstraction riêng)	Cửa sổ ngữ cảnh dài, phù hợp đọc đồng thời CV + JD + transcript phỏng vấn dài
Speech-to-Text	Whisper API hoặc AssemblyAI	Hỗ trợ speaker diarization (phân biệt người nói), chất lượng tốt với tiếng Việt
Triển khai (Deploy)	Docker + Docker Compose → Kubernetes khi mở rộng	Tách AI-worker thành service riêng để scale độc lập với API chính

2. CẤU TRÚC THƯ MỤC BACKEND (FASTAPI)

Với hệ thống có nhiều domain nghiệp vụ (JD, ứng viên, lịch phỏng vấn, AI...), cấu trúc modular theo domain (Modular/DDD-lite) sẽ dễ bảo trì hơn so với MVC thuần túy. Mỗi module tự chứa Router (Controller) - Schema (DTO) - Service (business logic) - Model (ORM).

backend/ — Cấu trúc Modular theo domain

```
backend/
├── app/
│   ├── main.py                # Khởi tạo FastAPI app
│   ├── core/
│   │   ├── config.py          # Settings (env, secrets)
│   │   ├── security.py         # JWT, hashing
│   │   └── database.py         # SQLAlchemy engine/session
│   ├── modules/               # Mỗi domain = 1 module doc lap
│   │   ├── auth/
│   │   │   ├── router.py      # (Controller) API endpoints
│   │   │   ├── schemas.py     # (Pydantic) DTO request/response
│   │   │   ├── service.py     # Business logic
│   │   │   └── models.py      # (Model) SQLAlchemy ORM
│   │   ├── job_posting/       # Module quản lý tin JD
│   │   │   ├── router.py
│   │   │   ├── schemas.py
│   │   │   ├── service.py
│   │   │   └── models.py
│   │   ├── candidate/         # Module ứng viên & CV
│   │   │   ├── router.py
│   │   │   ├── schemas.py
│   │   │   ├── service.py
│   │   │   ├── models.py
│   │   │   └── cv_parser.py   # Logic parse CV (gọi AI layer)
│   │   ├── interview/         # Module lịch phỏng vấn
│   │   │   ├── router.py
│   │   │   ├── schemas.py
│   │   │   ├── service.py
│   │   │   ├── models.py
│   │   │   └── calendar_sync.py # Google/Outlook Calendar
│   │   ├── evaluation/        # Module đánh giá sau PV
│   │   │   ├── router.py
│   │   │   ├── schemas.py
│   │   │   ├── service.py
│   │   │   └── models.py
│   │   ├── email/             # Module email hàng loạt
│   │   │   ├── router.py
│   │   │   ├── schemas.py
│   │   │   ├── service.py
│   │   │   └── templates/
│   └── ai/                    # Lớp AI riêng biệt
│       ├── llm_client.py       # Abstraction: gọi Claude/GPT không sửa nội khác
│       ├── prompts/
│       │   ├── cv_match.py     # Prompt chấm CV-JD
│       │   ├── interview_questions.py
│       │   └── interview_evaluation.py
```

```

├── embeddings.py          # Tao & so khop vector
├── stt.py                 # Speech-to-Text wrapper
├── schemas.py             # Structured output (JSON schema cho LLM)
├── workers/              # Celery tasks (xu ly nen)
│   ├── celery_app.py
│   ├── tasks_cv_scoring.py
│   ├── tasks_interview_analysis.py
│   └── tasks_bulk_email.py
├── shared/
│   ├── exceptions.py
│   ├── permissions.py    # Phan quyen theo Role
│   └── utils.py
├── alembic/              # Migration DB
├── tests/
│   └── modules/
│       └── ai/
├── requirements.txt
└── docker-compose.yml

```

Điểm mấu chốt: thư mục ai/ tách biệt hoàn toàn khỏi modules/. Mỗi module (candidate, interview...) chỉ gọi vào ai/llm_client.py chứ không tự viết prompt hay gọi API AI trực tiếp → sau này đổi model/provider chỉ cần sửa một chỗ duy nhất.

3. CẤU TRÚC THƯ MỤC FRONTEND (NEXT.JS)

frontend/ — Next.js App Router

```
frontend/
├── app/
│   ├── (public)/                # Công ung vien
│   │   ├── jobs/[slug]/page.tsx  # Trang chi tiet JD
│   │   └── apply/[jobId]/page.tsx # Form nop CV
│   ├── (dashboard)/             # App HR (can auth)
│   │   ├── jobs/
│   │   │   ├── candidates/
│   │   │   │   └── [id]/         # Chi tiet ung vien + diem AI
│   │   │   ├── interviews/
│   │   │   ├── evaluations/
│   │   │   └── reports/
│   │   └── layout.tsx
│   └── components/
│       ├── ui/                  # Button, Table, Modal... (design system)
│       ├── candidate/
│       │   ├── MatchScoreCard.tsx # Hien thi % phu hop + breakdown
│       │   └── CandidatePipeline.tsx
│       ├── interview/
│       │   ├── SchedulePicker.tsx
│       │   └── AIQuestionSuggestions.tsx
│       └── evaluation/
│           └── TranscriptViewer.tsx # Xem transcript + timestamp
├── lib/
│   ├── api/                    # Client goi backend theo module
│   ├── hooks/
│   └── utils/
└── types/                      # Type dung chung FE-BE
```

4. LUỒNG GỌI AI (KIẾN TRÚC TÁCH LỚP)

Toàn bộ lời gọi AI trong các module nghiệp vụ đều đi qua một lớp trung gian duy nhất, giúp việc đổi nhà cung cấp mô hình (Anthropic ⇌ OpenAI ⇌ mô hình mở) không ảnh hưởng đến logic nghiệp vụ.

Sơ đồ luồng xử lý AI

```
Module (candidate / interview / evaluation)
  |
  v
ai/llm_client.py  ----> Anthropic API / OpenAI API (doi duoc)
  |
  v
ai/prompts/*.py   (prompt template + structured output schema)
  |
  v
Celery worker (chay async, co retry, timeout)
  |
  v
Ghi ket qua vao DB --> tra ve cho FE qua polling/websocket
```

Lợi ích của thiết kế này

- Đổi model AI (ví dụ Claude → GPT-4 hoặc mô hình mở) chỉ cần sửa trong llm_client.py.
- Prompt được quản lý tập trung trong ai/prompts/, dễ version hóa và A/B test.
- Các tác vụ AI nặng (chấm CV, phân tích ghi âm) chạy nền qua Celery, không làm chậm trải nghiệm người dùng.
- Kết quả AI luôn được lưu vào DB trước khi trả về FE, đảm bảo có lịch sử để đối chiếu/audit sau này.

— Hết tài liệu —